
Fuzzing Lecture Material

Course: *Introduction to Vulnerability Research*

Author name: *Dr. Bernard Lampe*

Due date: *October 22nd, 2019*

Introduction

1. Miller et al.'s 1990 "An Empirical Study of the Reliability of UNIX Utilities": piping random garbage into standard UNIX utilities crashed or hung 25–33% of them, and roughly 9% of the programs that crashed did so on every input. Fuzzing was born as: throw structured-ish randomness at a parser and watch for SIGSEGV. The same study was repeated in 1995 ("Fuzz Revisited"), 2000, 2006 and 2020; the failure rate fell but never to zero, which is the point.
2. The most important questions are "When is using fuzzing appropriate/productive?" and "If fuzzing is appropriate, what type of fuzzing should be used?"
 - (a) Productive when: the target parses rich untrusted input (file formats, network protocols, IPC), input structure is deep (many nested layers), and there is time for corpus development. Poor when: input entropy is small (a config file with three booleans) or the interesting logic is behind crypto/compression you cannot traverse.
 - (b) Fuzzing has a hard time getting past encryption, hashing or compression algorithms. Usually patch out or fuzz afterwards: decrypt the payload first, or fuzz the plain-text post-processor. Compression is the same: decompress first, then fuzz the decompressed stream.
 - (c) The cost model is three-part: setup cost (build the harness and the corpus), per-execution cost (executions per second, the throughput that decides whether a campaign finishes), and triage cost (every crash must be minimized and root-caused). Fuzzing pays off only when the per-execution cost is small and the target parses many similar inputs.
3. Fuzzing has low false positives. A crash, hang, or sanitizer report is a real, observed deviation: reproduction is part of triage. The flip side is the oracle problem: a fuzzer only finds what the oracle can see, so bugs that do not crash or hang are invisible to it.
4. Fuzzing results in an unminimized input file that triggers a bug. The raw output of a campaign is a crash set, not a vulnerability report; the distance between the two is the triage pipeline below.
5. Outcomes (crashes, hangs) must be triaged first. Duplicates (same root cause, different input) dominate raw crash counts: dedupe on stack hash. A hang is usually an infinite loop or an expensive input; distinguish by sampling coverage.
6. Root causing can be hard. Minimize input first by bisecting the input (dd in a loop, delta, libFuzzer's `-minimize_crash=1`).

7. A minimized input then gets root-caused: map the crashing path in the debugger/source, identify the memory error class, evaluate exploitability. The full pipeline is taught in the **Crash Triage and Exploitability** section below.
8. Fuzzing is orthogonal to source auditing: it is the dynamic arm of analysis, and it is strongest exactly where source auditing is weakest (deep parser state that no one would read by hand) and weakest exactly where source auditing is strongest (semantic bugs, non-crashing logic errors, missed attack surface).

Fuzzing Theory

1. Dumb fuzzing, black box. No knowledge. Low penetration: mutations rarely satisfy magic values, checksums, and structural nesting, so most inputs die in early validation. Still useful as a cheap first pass.
2. Smart fuzzing, white box. Complete re-implementation of the parser. Max penetration, max human effort. A generator that emits well-formed documents from a grammar is “smart” but only explores the grammar’s own paths.
3. A mix of both is needed to hit unknown code paths in unexpected ways: structure-aware mutation (mutate inside a format the tool understands) is the practical sweet spot.
4. Fuzzing can be seen as following compiler design in reverse. (Lexical analysis, syntax analysis, semantic analysis, code generator, optimizer, assembly generation.) A parser validates in the same order the compiler front end does; each validation layer is a wall between the fuzzer and deep code. Generators that produce valid tokens for layer n but invalid semantics for layer $n+1$ penetrate exactly n layers.
5. Penetration depth is the formal version of that intuition: the fraction of random inputs that pass each successive validation layer. The mean over layers is the Fuzzing Book’s *opacity*; it predicts how many executions a random fuzzer needs, because the probability of reaching depth n is the product of the per-layer pass rates.
6. Fuzzing as a general search problem: maximize discovery of new states subject to a budget of executions. Greedy mutation (hill-climbing) vs. global search. Random mutation is a biased sampler over the input space; a coverage signal turns it into hill-climbing over the space of reached code.
7. Fuzzing as a maze analogy: inputs are paths through the maze of branches; coverage feedback is the cheese smell.
8. Fuzzing as a directed search problem (A^*): coverage feedback approximates an admissible heuristic for “makes progress toward unexecuted code”. Solvers (an actual A^* implementation is symbolic execution) are the exact version.
9. The search framing answers “when should fuzzing work”: the input space is effectively infinite, the set of bug-triggering inputs is finite but tiny, and every fuzzer is a sampler that only succeeds if the sample can land on the tiny set. That

is why a good corpus (a set of already-deep inputs) contributes more than a better mutation operator.

Mutation Fuzzing

1. Binary corruptors: take a valid input, apply bit flips, byte swaps, arithmetic increments, chunk splices, and re-emit. Radamsa is the canonical general-purpose corruptor; AFL mutates with the same philosophy but inside its feedback loop.
2. Strengths: zero understanding of the format required; finds bugs in code the author of a generator never thought to emit; recovers from format misunderstanding (no model, no model errors).
3. Weaknesses: checksums, magic values, and length prefixes form staircases of friction that dumb mutation cannot climb; penetration depth is bounded by the number of independent structural fields a random walk can satisfy.
4. The step size is the number of bytes changed per input: a small step size preserves structure but explores slowly, a large one destroys structure but moves far. Havoc mode in AFL stacks many small mutations in one input to get both.
5. When should a mutation fuzzer be used? When you have a corpus of valid inputs but no format documentation and limited engineering time; also as a baseline to measure smarter fuzzers against.

Generative Fuzzing

1. Grammar, generators: an explicit model of the input (BNF grammar, protocol state machine, format description like a Wireshark dissector or a Kaitai Struct file) samples or systematically emits inputs. Tools in this family: ANTLR, Domato, Dharma, Peach, Sulley/boofuzz, Fuzzilli.
2. Strengths: passes validation layers by construction, so reaching deep code is immediate; the model documents your understanding, and emitting edgy semantics (length fields that lie, zero counts, recursion depth attacks) is easy.
3. Weaknesses: bugs live in the gap between the model and the implementation. If your grammar is too correct, you never violate the parser's assumption. Generators must deliberately degrade plausibility, or they only test the code the model describes.
4. The model is also a liability: a grammar that is wrong in the same way as the parser misses the same bugs, and a grammar that is right costs as much to build as the parser did to write. The generative fuzzer's budget is human, not machine.
5. When should a generative fuzzer be used? When the format is documented or reversed, and the state space is too large for mutation alone (protocol stateful, nested containers).
6. Grammar mutation (Gramatron, Nautilus) is the hybrid: mutate the grammar or the derivation tree rather than the byte string, and keep a byte-level fuzzer for the leaves.

Instrumentation

1. Instrumentation is about how to detect a software deficiency.
2. Crashes: SIGSEGV/SIGBUS/SIGABRT/SIGILL/SIGFPE via a crash handler. Raw crashes are the cheap detector, but only fire on the luckiest memory corruptions: the write must land on unmapped or protected memory. Everything else is silent.
3. Hangs: wall-clock timeout per input. Distinguish from slow-but-terminating inputs by re-running with doubled timeouts; a hang that survives a much larger timeout is a bug, one that does not is a performance issue.
4. Sanitizers are the standard upgraded detector:
 - (a) ASan (AddressSanitizer): redzones around stack/heap/global allocations, quarantine for freed memory: catches OOB read/write, UAF, double-free at first byte of error instead of first crash. Verified demonstration: a heap overflow at `buf[4]` into a `malloc(4)` buffer is silent at `-O1` (dead store) but reported at `-O0` with a full “heap-buffer-overflow WRITE of size 1” report. Lesson: sanitizer visibility depends on the optimizer; test the harness at the shipped optimization level.
 - (b) MSan (MemorySanitizer): uninitialized memory reads. Requires an instrumented libc and complete initialization of everything the harness itself allocates, or it reports its own noise.
 - (c) UBSan (UndefinedBehaviorSanitizer): signed overflow, bad shifts, misaligned pointers, out-of-bounds array indexing (`-fsanitize=undefined`); traps instead of aborting with `-fsanitize-trap`.
 - (d) TSan (ThreadSanitizer): data races, for the concurrency bugs static analysis cannot see.
 - (e) KASan/KCSan for kernel targets (see Feedback Fuzzing).
5. Differential fuzzing: run two implementations of the same spec on the same input (two TLS stacks, two JS engines, two decompressors); a divergence in accepted output is a bug in at least one. Finds semantic bugs no sanitizer sees, and needs no crash oracle at all.
6. Difficult to instrument: closed binaries and exotic runtimes need hooking frameworks (Frida, DynamoRIO, Valgrind, Intel PIN). Static binary instrumentation of a stripped binary is slow and fragile; prefer source-level sanitizer builds whenever obtainable.

Crash Triage and Exploitability

1. A fuzzing campaign produces crash sets, not vulnerability reports. Triage converts crash sets into reports:
 - (a) **Dedupe**: group crashes by stack hash at the crash frame. Raw counts are dominated by copies of the same root cause (thousands of inputs hitting

- one sifted strcpy). Sanitizers print the shadow-memory type (heap-buffer-overflow / UAF / double-free / stack); group by type + stack. Tools: afl-collect, crashwalk, CASR.
- (b) **Minimize:** bisect the trigger file (dd, delta, libFuzzer -minimize_crash) until the shortest reproducing input. Fixed-point: minimization is done when further reduction changes the crash location.
 - (c) **Reproduce deterministically:** plain ASLR/pid/time/rng source control, same options on every replay (ASAN_OPTIONS etc). Non-deterministic crashes need per-race seeds until stable; mark the seed recipe in the report.
 - (d) **Root cause:** walk the minimized trigger under a debugger and map it to source (or disassembly): which check is missing/wrong? Express the root cause in one or two sentences; if you cannot, the minimization is not done. Compare classes against the known bug patterns (overflow, UAF, type confusion, uninitialized use) and name the class explicitly.
 - (e) **Assess exploitability** against the deployed mitigations (NX, canary, ASLR/PIE, RELRO, FORTIFY_SOURCE): what primitive does the crash actually grant (arbitrary write? partial, fixed-length, controlled source?), and what attacker preconditions does it need?
 - (f) **Write the report:** minimized reproducer + one-paragraph root cause + primitive statement + affected builds. The vuln-research deliverable is the report, not the fuzzer log.
2. Exploitability heuristics, in decreasing order of confidence that a crash converts to a bug:
 - (a) Controlled-length, controlled-content write inside a large heap region: scheduler-compatible, near-certainly exploitable (it satisfies the ideal heap vulnerability qualities: control of the allocation size, the overflowed data, and the overflow length).
 - (b) Controlled target address: near-arbitrary write, exploitable modulo CFI.
 - (c) Uncontrolled-length overflow (long memcpy): trivially DoS, exploitable only if a same/adjacent-size object has interesting metadata.
 - (d) Hang/overtime/slow-alloc: usually quality-of-life issues only, unless it models a resource-exhaustion primitive on a protocol-adjacent target.
 - (e) Check the call site of the corruption *and* the crash frame: ASan reports the first faulting access, not the bug. A UAF read in strcmp may be the shadow of a double-free bug set up two frames earlier.
 3. Deduplicate at root-cause level before investing hours: two crashes sharing a deduped stack but with distinct minimizing paths can still be one bug; two crashes with identical minimizing paths but different stacks can be two bugs of one class. The stack is evidence, not verdict.

Corpus Distillation

1. A fuzzer is only as good as its corpus. The mean opacity of the format determines how many mutations the seed set needs; a rich seed set of real-world samples

contributes more than algorithmic cleverness. A seed that reaches deep code does the work of thousands of random ones.

2. Sources: application test suites, documentation examples, Wireshark protocol sample captures, media sample repositories, vendor sample repos, user forums (“my file won’t open” posts are especially valuable), and previously captured fuzz corpora (AFL’s bundled testcases, the OSS-Fuzz seed corpora).
3. Distillation: dedupe (by content hash is naive; by coverage reached is the right metric), unionize the corpus (drop seeds that add no new coverage), repair (fix corrupt samples so they parse), and minimize each retained seed.
4. Coverage-guided merging: for each candidate seed, run once with coverage; keep it if it covers an edge no kept seed does. Result: the minimum corpus that still meets the coverage criterion — the standard afl-cmin operational definition. libFuzzer’s `-merge=1` is the same algorithm over a different coverage map.

Code Coverage

1. Coverage is the only cheap oracle for “is the fuzzer making progress”. Flavors (weakest to strongest):
 - (a) Function coverage: executed functions.
 - (b) Edge (branch) coverage: executed source-sink pairs of control flow — the standard metric (AFL edges, source-based `-fsanitize-coverage=trace-pc-guard`).
 - (c) Path coverage: every distinct path. Combinatorially explosive; approximated with counters or (unmaintained) path hashes.
 - (d) Data coverage: distinct values of interesting comparisons, e.g. bytes compared against magic constants. Red Queen-style cmplog/CMF instrumentation tracks these to bypass magic-value walls.
2. Coverage map: a bitmap hash of observed edges; every new edge triggers mutation recruiting in feedback fuzzers. The map is the interface between the instrumented target and the fuzzing loop, and its collisions (two edges hashing to one slot) are why AFL uses a randomized location seed.
3. Measurement tools: `gcov/llvm-cov/lcov` for source-level line and branch coverage, `afl-showmap` and `afl-cmin` for AFL maps, `sancov` for the sanitizer-coverage hooks, and `kcov` for the kernel.
4. Coverage is a proxy, not a goal: a fuzzer can cover a line without exercising the state that makes it buggy. That gap is exactly the difference between the static and dynamic analysis families.

Feedback Fuzzing

1. AFL: pure feedback loop. Compile target with instrumentation, save seeds, mutate, run, compare edge bitmap against all previous runs, save as a new seed on novel coverage. In-process execution (fork server) raised throughput from $\sim k$ execs/s (process-per-case) to $\sim 10k+$ on trivial parsers.

2. libFuzzer: same feedback idea, in-process, linking the harness directly, with LLVMFuzzerTestOneInput. Faster than AFL but any crash takes down the fuzzer (mitigated by `-fork=1`). Knobs worth knowing: `-max_len`, `-timeout`, `-rss_limit_mb`, `-runs`, `-dict`, `-jobs/-workers`, `-merge=1`, `-minimize_crash=1`, and the `LLVMFuzzerInitialize/LLVMFuzzerCustomMutator` hooks.
3. honggfuzz: hardware counters (`perf_event`) for coverage; useful when recompilation is impossible. Its sanitizer integration is the reason it is used on binaries that cannot be rebuilt with coverage hooks.
4. AFL++: the modern fork of AFL. Persistent mode (`__AFL_LOOP`), `AFL_LLVM_LAF_ALL` and `AFL_LLVM_CMPLOG` are the features that make it competitive on magic-value-heavy formats.
5. KASAN: kernel AddressSanitizer. Pairs with syzkaller's sys-CALL descriptions to fuzz the syscall surface of a kernel: kernel-side feedback fuzzing end to end (the standard kernel fuzzers are syzkaller, kAFL/QAFL, Trinity, and razer).

Concolic and Symbolic Fuzzing

1. Symbolic execution: input values become symbols; path constraints accumulate; a solver (SMT, e.g. Z3, STP, Boolector) produces inputs to reach chosen branches. Complete for straight-line code; incomplete under loops, floating point, syscalls, and state explosion.
2. Concolic = concrete + symbolic: run for real (concrete), collect path constraints, negate one, solve for the new input, repeat. AFL's coverage feedback is a crude approximation; full systems (KLEE, S2E, angr, Manticore, Triton) are exact but slow (100–1000x slowdown).
3. Practical use: hybrid. Fuzz for breadth, then symbolically solve specific friction points (magic string check, checksum loop) and inject the solved values back into the corpus (Red Queen / Driller / QSYM / SymCC lineage).
4. The symbolic arm is the exact version of the A* analogy from the theory section: it computes the input that reaches a chosen branch instead of guessing it. That is why it is the right tool for the checksum/magic staircase and the wrong tool for a deep stateful parser.

Scaling Fuzzing Issues

1. Reproducibility/Determinism: ASLR, PID/uid leakage, time- and randomness-dependent branches, and multithreading all make a crash non-replayable. Standard mitigations: `setarch -R` to disable ASLR (Linux), `ASAN_OPTIONS=abort_on_error=1:handle_a` for replayable aborts, single-threaded corpus mode, fixed time source, and a seed-stable PRNG. Keep the fuzzer's environment identical for replay (same kernel, same options: sanitizers read `ASAN_OPTIONS`).
2. Throughput vs. fidelity: in-process fuzzing is 100x faster but crashes take the host down, leak memory, and poison the corpus with stateful residue. Restart fuzzers periodically; `-fork=N` is the production answer.

3. Stats and hygiene: track executions/second, coverage growth, and crash rate; a flat coverage graph for hours means the fuzzer is stuck (new seeds or a new harness needed, not more cores).
4. Distributed fuzzing: send chunks of corpus to machines running the same fuzzer build over the same target with a shared corpus sync (AFL's -M/-S secondary-node scheme, and libFuzzer clusters): coverage merges at sync intervals. `afl-whatsup` summarizes the fleet.

Fuzzing Deeper

1. Theory about skipping outer layers: outer layers (network stack, container formats, compression scripts) consume fuzzing budget validating structure instead of exercising interesting parse state. When the outer layer is well-fuzzed, patch or emulate it: feed fuzzed data directly into the layer-*N* parser (the “structure-aware deep fuzzing” idea).
2. Techniques: split the target into its layers and harness each parser in isolation; stub out decompression; provide torn packets only at the boundary under test. The harness is where the engineering goes, not the mutator.
3. Example: fuzzing a media container. Rather than mutating whole files, generate valid container structure with fuzzed field values; or wrap the media parser so that the `demux_open()` entry is called with a fuzzed in-memory buffer, skipping the file layer.

Domain-Specific Fuzzing

1. Media fuzzing: image libraries (libpng, libjpeg, FFmpeg family). Historical bug source of CVEs; high value because images arrive from untrusted senders automatically (MMS, browsers, thumbnails). FFmpeg's own tools (`ffmpeg -f lavfi`), the last 10 years of oss-fuzz artifacts, and format-specific mutators (PDF mutators, media seed corpora from the Fuzzing Book) support this.
2. JavaScript (AST Fuzzing): language fuzzers mutate at the AST level (Fuzzilli's coverage-guided engine, Domato's DOM grammar), because text-level mutation of source produces syntax errors, not semantic depth. JIT compilers are the bug-rich frontier (JIT engines are a classic back-end component to prioritize in an audit).
3. Fuzzing network protocols (HTTP, HTTPS, DHCP, NTP, DNS, ...): stateless ones (DNS, DHCP) fuzz easily with packet generators; stateful ones (TLS, HTTP/2) need state machines and are where generative + feedback hybrids (TLS-Attacker, boofuzz, h2fuzz) were developed. Practical concern: fuzz the protocol decoder of the server, not the cryptography. That last assertion is a design decision, not a discovery.
4. Fuzzing other stuff: file systems (disk image fuzzing against fsck), firmware binaries (context-aware emulation with Firmadyne/FirmAE/QEMU), kernel syscalls (syzkaller), USB stacks, and the network stacks of embedded devices.

Open Source Fuzzers

1. Radamsa (black-box mutator, easy to pipe into scripts), AFL / AFL++ (feedback, fork server), libFuzzer (in-process, LLVM), honggfuzz (perf_event hardware-counter coverage for targets you cannot recompile), syzkaller (kernel syscalls), Trinity (syscall random caller), kAFL/QAFL (KVM/QEMU hardware-assisted feedback for firmware and closed targets), bochspwn-style instrumented emulators, OSS-Fuzz (Google's continuous fuzzing infrastructure for open source — many CVEs, ~thousands of bugs/month scale at peak).
2. Newer entrants worth naming: Centipede (Google's batched, sharded fuzzer), LibAFL (the Rust framework), Jazzer and go-fuzz/atheris for managed runtimes, and FuzzBench/Magma for measuring a fuzzer rather than a target.

Listing 1: Demo harness: libFuzzer entry point plus minimal standalone driver. Built clang -g -O1 -fsanitize=address; both parts verified on this host.

```
1 #include <stdint.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size)
6 {
7     if (size < 5) return 0;
8     char *buf = malloc(4);
9     if (memcmp(data, "FUZZ", 4) == 0 && data[4] >= '\0') {
10         /* BUG: attacker-controlled length into a 4-byte allocation */
11         memcpy(buf, data, size);
12     }
13     free(buf);
14     return 0;
15 }
```

Listing 2: Standalone driver for running the same harness on a single file (verified: triggers ASan “heap-buffer-overflow” at -O0, silent at -O1).

```
1 #include <stdio.h>
2 #include <stdint.h>
3 #include <stdlib.h>
4
5 int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size);
6
7 int main(int argc, char **argv)
8 {
9     if (argc != 2) return 1;
10    FILE *f = fopen(argv[1], "rb");
11    if (!f) return 1;
12    static uint8_t buf[4096];
13    size_t n = fread(buf, 1, sizeof(buf), f);
14    fclose(f);
15    LLVMFuzzerTestOneInput(buf, n);
16    return 0;
17 }
```

Conclusion

Fuzzing gives you a small number of high-quality, hard-to-discover memory-corruption bugs at the cost of setup and infrastructure. It is orthogonal to source auditing: its weak spot is precisely what source auditing is strong at (non-crashing logic bugs, semantic errors, missed attack surface), and vice versa (static and dynamic analysis win on different bug classes). The craft is in the corpus, the harness, and the detection layer, not in mutating bytes: budget your fuzzing time accordingly.

The same conclusion, stated as a rule of thumb: fuzzing is productive when the target parses rich untrusted input, the harness reaches the parser directly, and the oracle (sanitizer, differential, or crash) can see the bug class you care about. When any one of those three is missing, the campaign is a random-number generator with a coverage report.

References

1. www.fuzzingbook.com
2. AFL: lcamtuf.coredump.cx/afl/
3. libFuzzer: llvm.org/docs/LibFuzzer.html
4. Radamsa: gitlab.com/akihe/radamsa
5. syzkaller: github.com/google/syzkaller
6. kAFL: github.com/RUB-SysSec/kAFL
7. oss-fuzz: github.com/google/oss-fuzz

8. Miller et al., “An Empirical Study of the Reliability of UNIX Utilities”, CACM 1990
9. AFL++: github.com/AFLplusplus/AFLplusplus
10. FuzzBench and Magma: github.com/google/fuzzbench, github.com/HexHive/magma
11. Fuzzilli: github.com/googleprojectzero/fuzzilli